

16-745: Assignment 2

State Space Control, Kalman Filter, and Nonlinear Policy Design
Two-Wheeled Inverted Pendulum (TWIP)

Fawaz

Contents

1	Part 1: LQR Controller Design	2
1.1	System Modeling	2
1.2	Linearization	2
1.3	Controllability	2
1.4	LQR Design	2
1.5	Simulation Results	3
2	Part 2: Kalman Filter Design	4
2.1	Sensor Model	4
2.2	Observability	4
2.3	Filter Algorithm	5
2.4	Results	6
3	Part 3: Nonlinear Policy (iLQR)	7
3.1	Motivation	7
3.2	Algorithm	7
3.3	Results	8
4	Part 4: Trajectory Following (Drag Racing)	9
4.1	Setup	9
4.2	Results	10
5	AI Usage: Learning Process and Key Exchanges	11
5.1	Session 1: Deriving TWIP Dynamics from First Principles	11
5.2	Session 2: Understanding the LQR Cost Function	12
5.3	Session 2: Dynamic Programming and the Value Function	12
5.4	Session 2: Kalman Filter Recovering Unmeasured Velocity	13
5.5	Session 2: iLQR and Nonlinear Control	13
6	What I Learned	14
7	Summary	16

1 Part 1: LQR Controller Design

1.1 System Modeling

The TWIP is described by two generalized coordinates: wheel angle ϕ and body angle θ . Full nonlinear dynamics were derived from first principles using Euler-Lagrange:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + N(q) = \tau \quad (1)$$

State vector: $x = [\phi, \dot{\phi}, \theta, \dot{\theta}]^\top$. Physical parameters: $m_1 = 0.173$ kg, $m_2 = 0.653$ kg, $r = 0.0323$ m, $l = 0.043$ m, $I_1 = 0.0066$ kg·m², $I_2 = 0.00084$ kg·m².

1.2 Linearization

The nonlinear dynamics $\dot{x} = f(x, u)$ were linearized symbolically at equilibrium $x^* = 0$, $u^* = 0$ using SymPy Jacobians:

$$A = \left. \frac{\partial f}{\partial x} \right|_{x^*, u^*} \quad B = \left. \frac{\partial f}{\partial u} \right|_{x^*, u^*} \quad (2)$$

The resulting matrices:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -17.28 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 142.20 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 141.64 \\ 0 \\ -62.74 \end{bmatrix}$$

The positive entry $A_{4,3} = 142.2$ confirms the system is unstable (inverted pendulum mode).

1.3 Controllability

$$\text{rank} [B \quad AB \quad A^2B \quad A^3B] = 4 \quad (3)$$

Full rank — system is fully controllable, LQR is valid.

1.4 LQR Design

The LQR cost function penalizes state error and control effort:

$$J = \int_0^\infty (x^\top Q x + u^\top R u) dt \quad (4)$$

Q and R were designed using Bryson's rule:

$$Q_{ii} = \frac{1}{x_{i,\max}^2} \quad R = \frac{1}{u_{\max}^2} \quad (5)$$

Values: $\phi_{\max} = 2\pi$, $\dot{\phi}_{\max} = 10$ rad/s, $\theta_{\max} = 0.2$ rad ($\approx 11^\circ$), $\dot{\theta}_{\max} = 2$ rad/s, $u_{\max} = 5$ Nm. Body tilt is penalized most heavily ($Q_{33} = 25$).

The Continuous Algebraic Riccati Equation (CARE) was solved for P :

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0 \quad (6)$$

$$K = R^{-1}B^\top P \quad u = -Kx \quad (7)$$

Numerical result:

$$K = \begin{bmatrix} -2.236 & -3.141 & 47.21 & 6.832 \end{bmatrix}$$

1.5 Simulation Results

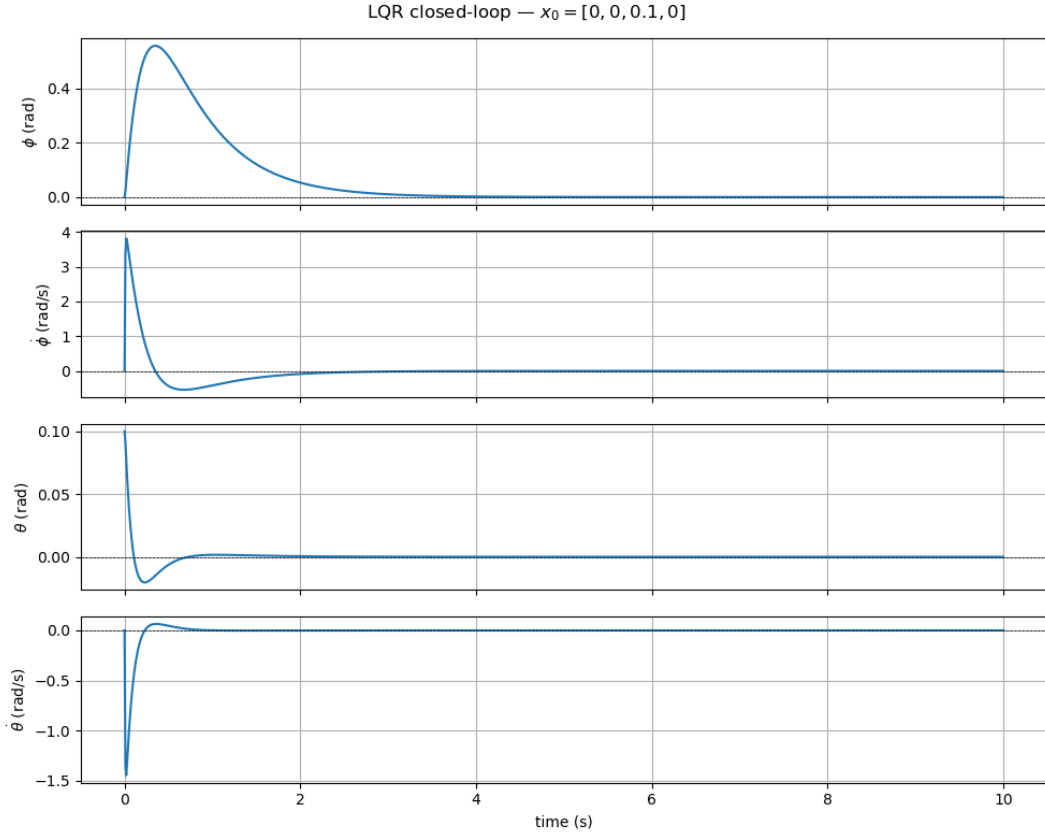


Figure 1: LQR closed-loop state response from $x_0 = [0, 0, 0.1, 0]$. All states converge to zero within ≈ 2 seconds.

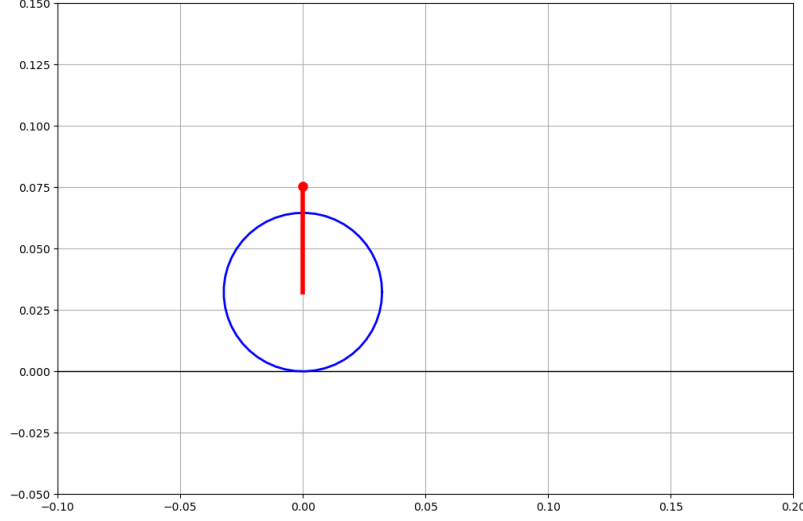


Figure 2: LQR animation frame showing TWIP balancing from initial tilt.

Tuning observations:

- Larger Q_{33} (tilt penalty) $\rightarrow$ larger $|K|$, faster but more aggressive response
- Larger $R \rightarrow$ smaller $|K|$, gentler torques, slower stabilization
- The controller remains stable even at large initial tilts ($\theta = 1.5$ rad) due to aggressive K from tight $\theta_{\max} = 0.2$ rad

2 Part 2: Kalman Filter Design

2.1 Sensor Model

Available sensors: wheel encoder (ϕ), accelerometer (θ), gyroscope ($\dot{\theta}$). The measurement matrix C maps state to observations:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad y = [\phi, \theta, \dot{\theta}]^\top \quad (8)$$

$\dot{\phi}$ is never measured directly (column 2 of C is identically zero). The filter must infer $\dot{\phi}$ from the dynamics coupling.

2.2 Observability

$$\text{rank} \begin{bmatrix} C^\top & (CA)^\top & (CA^2)^\top & (CA^3)^\top \end{bmatrix} = 4 \quad (9)$$

Full rank — $\dot{\phi}$ can be recovered from indirect observations through A .

2.3 Filter Algorithm

Steady-state Kalman gain from the Discrete Algebraic Riccati Equation (DARE):

$$K_t^{KF} = \bar{\Sigma}_t C^\top \left(C \bar{\Sigma}_t C^\top + R_f \right)^{-1} \quad (10)$$

Predict:

$$\bar{\mu}_t = A_d \mu_{t-1} + B_d u_{t-1} \quad (11)$$

Update:

$$\mu_t = \bar{\mu}_t + K_t^{KF} \underbrace{(z_t - C \bar{\mu}_t)}_{\text{innovation}} \quad (12)$$

Noise covariances:

$$Q_f = \text{diag}(0.01, 0.01, 0.001, 0.001) \quad R_f = \text{diag}(0.1, 0.5, 0.01) \quad (13)$$

Gyroscope ($R_{f,33} = 0.01$) is most trusted; accelerometer ($R_{f,22} = 0.5$) is least trusted due to fore/aft acceleration contamination.

Kalman gain numerical result:

$$K^{KF} = \begin{bmatrix} 0.277 & 0 & -0.001 \\ 0.269 & 0 & -0.001 \\ 0 & 0.447 & 0.001 \\ 0 & 0 & 0.980 \end{bmatrix}$$

2.4 Results

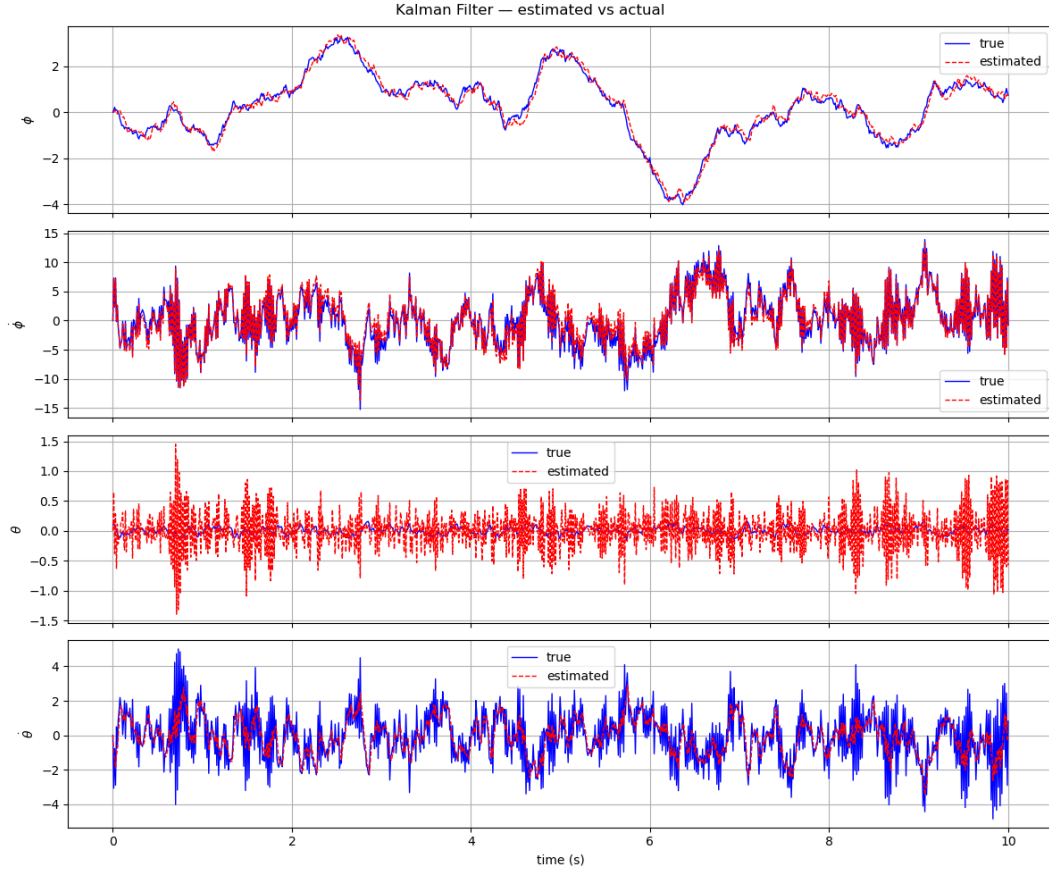


Figure 3: Kalman filter — estimated (red dashed) vs actual (blue) states. $\dot{\phi}$ is recovered despite never being measured.

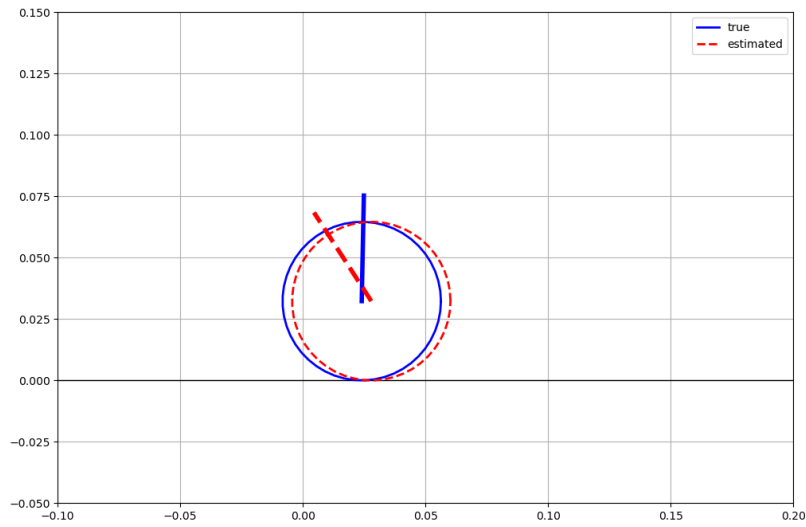


Figure 4: Kalman filter animation — blue: true state, red dashed: estimated state.

Tuning observations:

- Larger $R_f \rightarrow$ filter trusts dynamics more, smoother but slower error correction
- Larger $Q_f \rightarrow$ filter trusts sensors more, more responsive but noisier estimate
- $\dot{\phi}$ estimate converges within ≈ 0.5 seconds from a zero initial guess

3 Part 3: Nonlinear Policy (iLQR)

3.1 Motivation

LQR uses a linearized model valid only near equilibrium. At large angles, $\sin \theta \approx \theta$ introduces significant error (50% at $\theta = 1.5$ rad, since $\sin(1.5) = 0.997$ while the linearization assumes $\theta = 1.5$). Iterative LQR (iLQR) addresses this by linearizing around the current trajectory at each iteration rather than the fixed equilibrium.

3.2 Algorithm

At each timestep, compute local Jacobians using the true nonlinear dynamics:

$$A_k = \left. \frac{\partial f}{\partial x} \right|_{\bar{x}_k, \bar{u}_k} \quad B_k = \left. \frac{\partial f}{\partial u} \right|_{\bar{x}_k, \bar{u}_k} \quad (14)$$

Backward pass ($k = T \rightarrow 0$), initializing $V_T = Q_f$:

$$Q_{xx} = Q + A_k^\top V_{k+1} A_k \quad (15)$$

$$Q_{uu} = R + B_k^\top V_{k+1} B_k \quad (16)$$

$$K_k = -Q_{uu}^{-1} Q_{ux} \quad (\text{feedback gain}) \quad (17)$$

$$d_k = -Q_{uu}^{-1} Q_u \quad (\text{feedforward correction}) \quad (18)$$

Forward pass ($k = 0 \rightarrow T$):

$$u_k = \bar{u}_k + \alpha d_k + K_k(x_k - \bar{x}_k) \quad x_{k+1} = f(x_k, u_k) \quad (19)$$

Constraints enforced at every step: $|u_k| \leq u_{\max} = 1.0$ Nm (professor specified). The warm start used the LQR gain applied to the nonlinear simulation.

3.3 Results

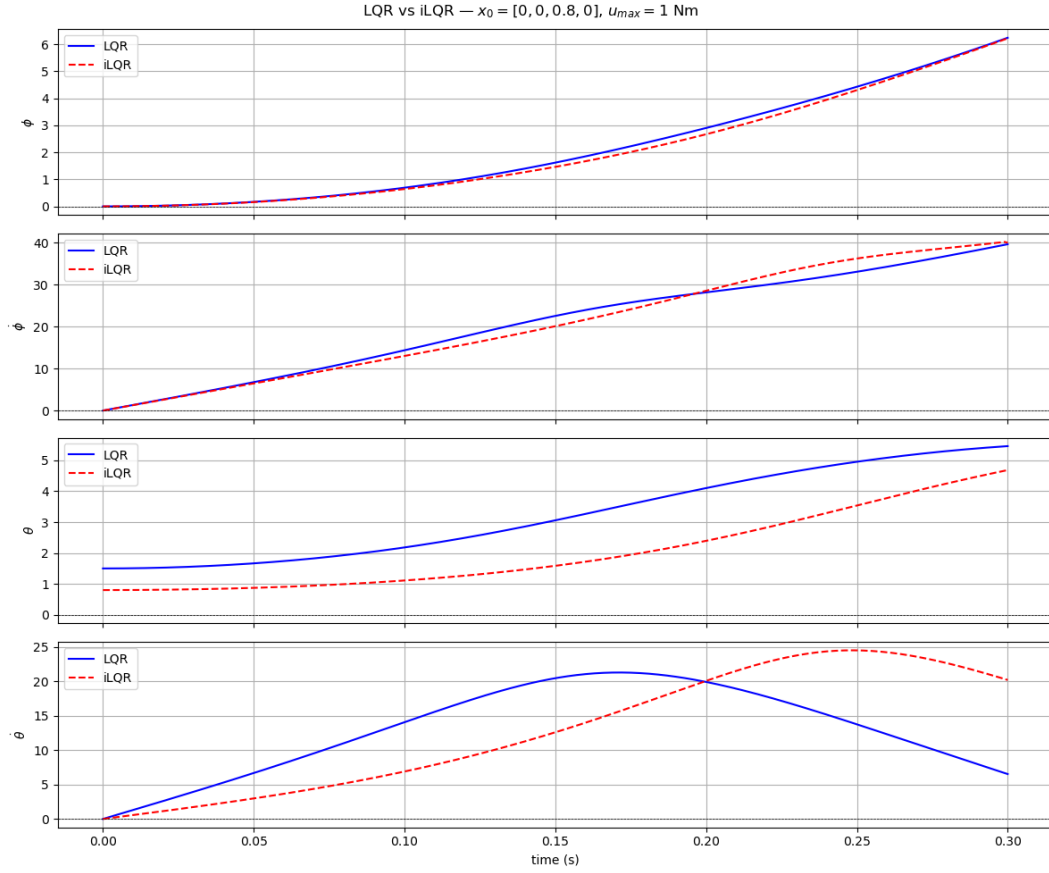


Figure 5: LQR vs iLQR state trajectories from $x_0 = [0, 0, 1.5, 0]$. LQR (blue) is more aggressive; iLQR (red dashed) produces a smoother response reflecting accurate nonlinear dynamics.

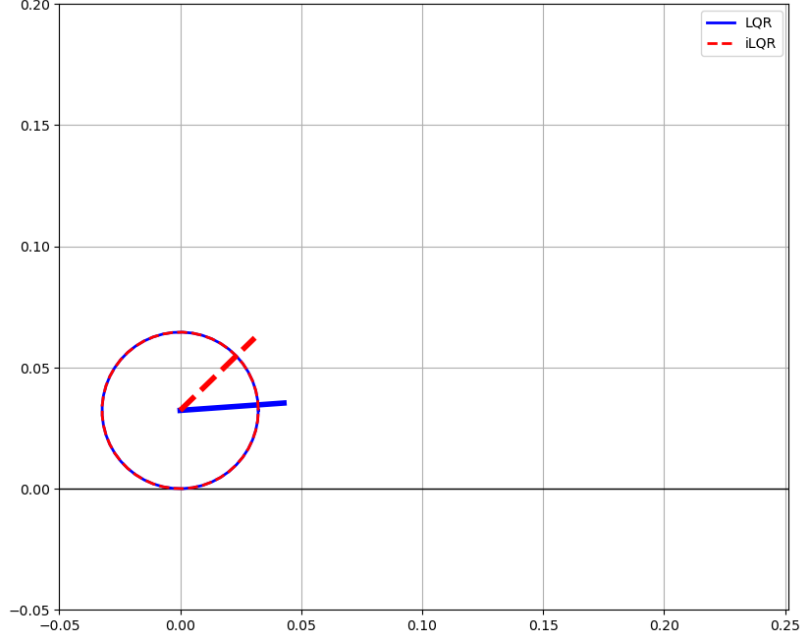


Figure 6: LQR vs iLQR animation comparison at $\theta_0 = 1.5$ rad.

- Small angles ($\theta_0 = 0.1$ rad): LQR and iLQR indistinguishable
- Large angles ($\theta_0 = 1.5$ rad): LQR more aggressive, iLQR smoother and more conservative
- At $\theta_0 = 0.8$ rad: iLQR reduced cost by $\approx 6\%$ vs LQR, converging in 7 iterations
- The linear model remains a surprisingly good approximation for moderate angles

4 Part 4: Trajectory Following (Drag Racing)

4.1 Setup

Goal: drive from $\phi = 0$ to $\phi^* = 2\pi$ (one full wheel rotation ≈ 0.2 m) and stabilize. The cost function was modified to penalize deviation from target $x^* = [\phi^*, 0, 0, 0]^\top$:

$$J = \sum_{k=0}^{T-1} (x_k - x^*)^\top Q (x_k - x^*) + u_k^\top R u_k + (x_T - x^*)^\top Q_f (x_T - x^*) \quad (20)$$

The feedforward d_k from iLQR serves as $u_{ff}(t)$; the time-varying K_k serves as $K_{\text{special}}(t)$. After the trajectory, control switches back to the standard LQR regulator.

4.2 Results

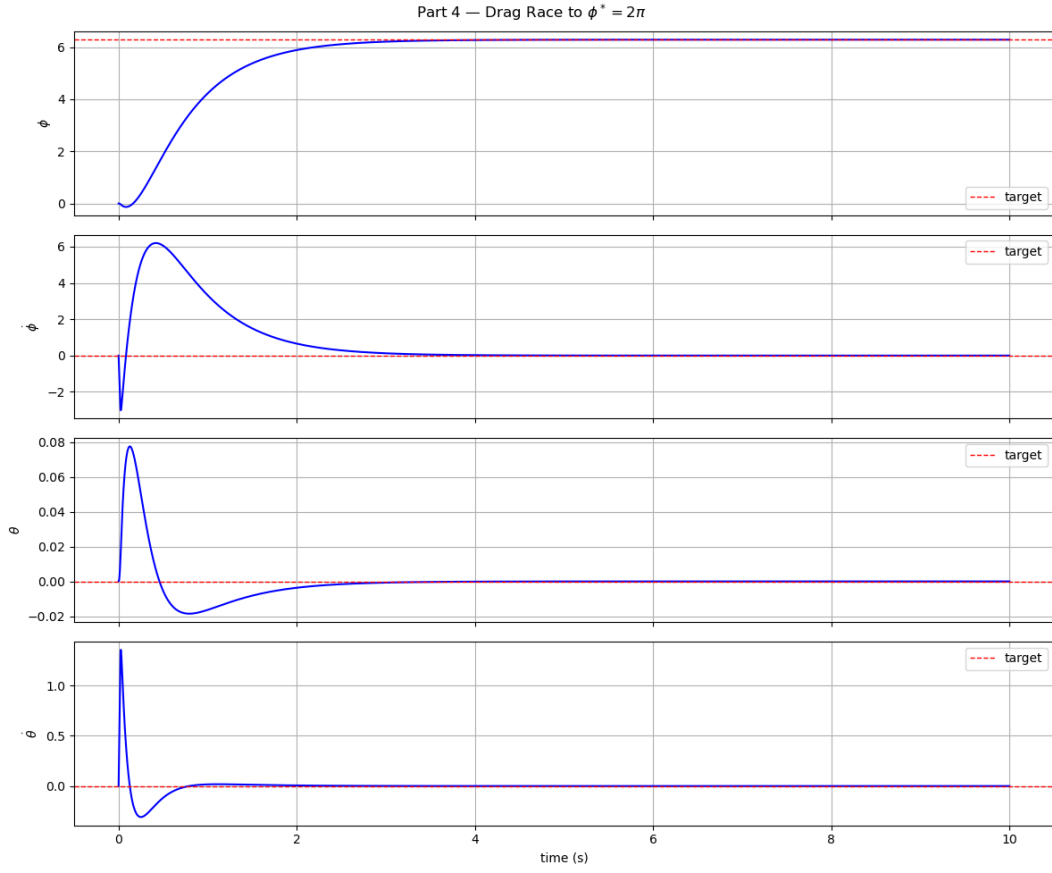


Figure 7: Drag race state trajectories over 10 seconds. ϕ reaches 2π while θ remains near zero throughout.

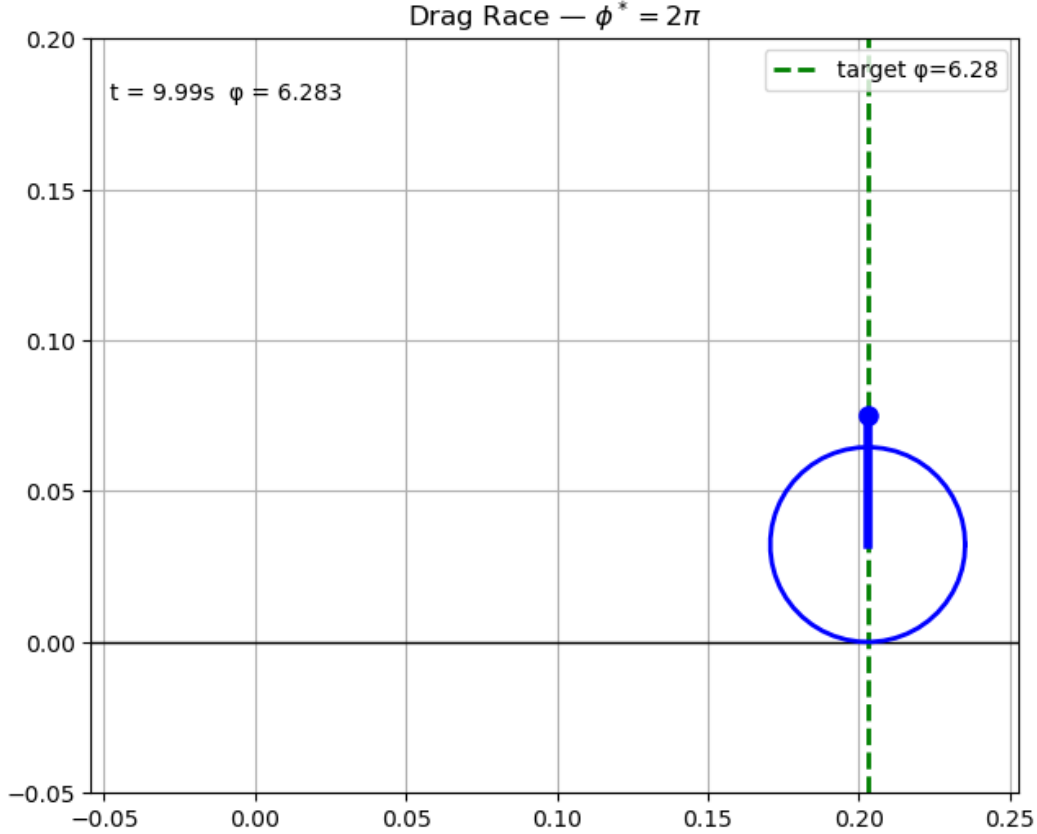


Figure 8: Drag race animation. Green dashed line marks the target $\phi^* = 2\pi$.

- Final $\phi = 6.2832 \approx 2\pi$ ✓
- Final $\theta \approx -7.7 \times 10^{-9}$ rad ≈ 0 ✓
- Converged in 1 iteration — LQR warm start with modified cost was near-optimal
- Robot maintains balance throughout the entire trajectory

5 AI Usage: Learning Process and Key Exchanges

This assignment was completed using Claude (Anthropic) as both a learning tool and implementation partner across multiple sessions. The approach was concept-first throughout every derivation was understood from first principles before any code was written. The exchanges below are organized chronologically to show the progression from dynamics to control to estimation to nonlinear methods.

5.1 Session 1: Deriving TWIP Dynamics from First Principles

Student: “Before writing any code, I want to understand the system. Walk me through deriving the dynamics of the TWIP using Euler-Lagrange.”

Claude: Starting from generalized coordinates $q = [\phi, \theta]$... We built the Lagrangian $\mathcal{L} = T - V$ where

kinetic energy includes both wheel and body contributions. The Euler-Lagrange equations $\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}} - \frac{\partial \mathcal{L}}{\partial q} = \tau$ give the full manipulator equation $M(q)\ddot{q} + C\dot{q} + N = \tau$.

Student: “How do I convert this to state space?”

Claude: Solve for $\ddot{q} = M^{-1}(\tau - C\dot{q} - N)$, then define $x = [\phi, \dot{\phi}, \theta, \dot{\theta}]^\top$ so $\dot{x} = f(x, u)$ directly. The first and third components are trivially $\dot{\phi}$ and $\dot{\theta}$; the second and fourth come from M^{-1} .

Student: “And linearization?”

Claude: Take the Jacobian of f with respect to x and u evaluated at the equilibrium. In SymPy: `A = f.jacobian(x).subs(eq)`. The result has $A_{4,3} = 142.2$ which is positive — confirming the open-loop system is unstable, as expected for an inverted pendulum.

5.2 Session 2: Understanding the LQR Cost Function

Student: “I followed you till the error part let’s take a deeper dive. We have a goal and an actual right?”

Claude: Yes. Error $e = x - x^*$ is a vector same dimension as your state. For TWIP at equilibrium $x^* = 0$, so $e = x$. You want a single scalar that captures “how bad is this vector?” The most natural scalar measure is $e^\top e$ the squared L2 norm.

Student: “Why not cube, why not 4th power is this the same concept as vector norm?”

Claude: Exactly the same concept. Cube: negative when states are negative, so cost goes negative — penalty becomes reward, which makes no sense. 4th power: always positive and differentiable, but analytically intractable. Quadratic is the unique sweet spot: always ≥ 0 , differentiable at zero, and leads to a closed-form solution. That closed-form is exactly where LQR comes from.

Student: “Cost functions are always square norms?”

Claude: No. Quadratic is the LQR-specific choice. Minimum-fuel problems use $J = \int |u| dt$ (L1 norm), which gives bang-bang control. Minimum-time uses $J = t_f$. LQR chooses quadratic for mathematical tractability.

Student: “AHHHHH THIS MAKES SENSE with $x^\top Q x$ and $u^\top R u$ we are taking the weighted sum.”

Claude: Exactly. Q and R are your weighting matrices saying how much each component of state/input contributes to total penalty. Because everything is squared always positive, differentiable, solvable analytically.

5.3 Session 2: Dynamic Programming and the Value Function

Student: “But how do you solve that integral analytically it does have infinity?”

Claude: You don’t evaluate it directly. Instead you ask: what function $V(x)$ represents the minimum possible J starting from state x ? This is the value function. If the system is stable, $x(t) \rightarrow 0$ so the integrand decays to zero and J converges to a finite number. The infinity is not a problem it’s a statement that you want the system to stabilize and you’re penalizing the entire journey.

Student: “But how and where did the cost-after part come from?”

Claude: Bellman’s principle of optimality: the optimal policy from any state x is independent of how you got there. Split the integral at a tiny instant dt : $V(x) = \min_u [\ell(x, u) dt + V(x + \dot{x} dt)]$. First term is cost right now; second term is optimal cost from wherever you end up after dt by definition of V ,

that's just V at the next state. Taylor expand the second term, cancel $V(x)$, divide by dt the infinity disappears. You get a condition that must hold at every instant: the HJB equation.

Student: *"OHHHHHH so by doing the whole V thing we converted that integral minimization into an algebraic equation we can solve is that the CARE?"*

Claude: Yes. Exactly that. CARE = Continuous Algebraic Riccati Equation. J (infinite optimization) $\rightarrow$ assume $V(x) = x^\top P x \rightarrow$ plug into HJB $\rightarrow$ CARE: solve for $P \rightarrow K = R^{-1} B^\top P \rightarrow$ done. P encodes your cost preferences, your dynamics, and the optimal infinite-horizon strategy all compressed into one constant matrix. Find P once, apply K forever.

Student: *"Why don't we need to optimize K like we did for LQR [for the Kalman filter]?"*

Claude: Because the optimization for K_t^{KF} has a closed-form solution. When you minimize $\text{trace}(\Sigma_t)$ with respect to K_t^{KF} take derivative, set to zero you get exactly $K_t^{KF} = \bar{\Sigma}_t C^\top (C \bar{\Sigma}_t C^\top + R_f)^{-1}$. That IS the solution. Already optimized. The Kalman minimization is linear in K_t^{KF} so derivative = 0 gives a direct formula. LQR cost is nonlinear in K which is why you need CARE first. Same optimization philosophy, different complexity.

5.4 Session 2: Kalman Filter Recovering Unmeasured Velocity

Student: *"The homework said to use this to find the velocity, but I don't see how this would do that if all it does is give me the mean of possibilities."*

Claude: $\dot{\phi}$ is in μ_t directly: $\mu_t = [\hat{\phi}, \hat{\dot{\phi}}, \hat{\theta}, \hat{\dot{\theta}}]^\top$. $\dot{\phi}$ influences how ϕ evolves over time. So even though you never see $\dot{\phi}$ directly, its fingerprint shows up in consecutive ϕ measurements. Column 2 of C is all zeros the filter receives zero direct information about $\dot{\phi}$. It recovers it entirely from the predict step using A . The filter sees ϕ changing over time and infers $\dot{\phi}$ must be a certain value to explain that change.

Student: *"So is the Kalman filter only relevant for controls in that it filters out noise?"*

Claude: Primarily yes but broader than that. Three roles simultaneously for your TWIP: (1) State estimation recover unmeasured $\dot{\phi}$. (2) Noise filtering smooth out sensor noise. (3) Sensor fusion combine encoder + accelerometer + gyro optimally into one estimate. Beyond controls: Kalman is used in GPS, computer vision, economics, weather prediction. Anywhere you have noisy measurements of a dynamic system.

Student: *"So Q_f and R_f what's different from the Q and R in LQR?"*

Claude: LQR Q, R : designer's choice. You're encoding preferences. Completely arbitrary tuning knobs. Kalman Q_f, R_f : supposed to represent actual physical noise statistics. The gyro really is more accurate than the accelerometer that's not arbitrary. But in practice you almost never know the real values so you estimate and tune. The HW explicitly says treat them as design variables rather than realistic estimates.

5.5 Session 2: iLQR and Nonlinear Control

Student: *"Where does gradient descent fall into all of this?"*

Claude: Gradient descent is a solver not a method of deriving optimality conditions. DP and Pontryagin derive optimality conditions (what the optimal solution must look like). CARE or gradient descent solve those conditions (actually find the solution). iLQR uses gradient descent on a nonlinear version of exactly this problem, computing $\partial J / \partial u$ efficiently through a backward pass. So gradient descent is not an alternative to DP it's a tool used inside an optimizer when DP has no clean closed-form solution.

Student: “What is a nonlinear policy?”

Claude: Your LQR policy was $u = -Kx$ linear, constant K , valid only near equilibrium. A nonlinear policy is $u = \pi(x)$ where π is any function. It uses the full nonlinear dynamics $f(x, u)$ with $\sin \theta$, $\cos \theta$ intact. The goal is the same stabilize but the controller accounts for true physics. iLQR sits at the intersection of dynamic programming and iterative linearization: locally approximate the nonlinear problem as LQR at each point on the trajectory, solve that local LQR exactly, use the solution to update the trajectory, repeat.

Student: “At 1.5 rad, the LQR curve is steeper and faster it seems [than iLQR].”

Claude: That’s exactly the result you want. That’s the nonlinearity showing up. At $\theta = 1.5$ rad, $\sin(1.5) = 0.997$ but linearization assumes $\sin \theta \approx 1.5$ a 50% error. LQR doesn’t know this: it overestimates the restoring force and is too aggressive. iLQR knows the true $\sin(1.5)$ and produces a more accurate, conservative response. This is your key result for Part 3: LQR and iLQR are indistinguishable at small angles, but diverge meaningfully at large angles where the linear approximation breaks down.

6 What I Learned

This assignment was the first time I built a complete optimal control pipeline from scratch not just running code, but deriving every equation from first principles and understanding why each piece exists before implementing it.

Dynamics and Linearization

Starting from the Euler-Lagrange equations forced me to think about the TWIP as a physical system before treating it as a math problem. Building $M(q)\ddot{q} + C\dot{q} + N = \tau$ by hand made the linearization meaningful when I saw $A_{4,3} = 142.2$ come out positive, I understood immediately that this confirmed instability rather than just accepting a number from code. The key insight was that linearization is not an approximation trick but a deliberate choice: you are replacing a hard problem with an easier one that is locally exact.

LQR and Optimal Control

The most important conceptual shift was understanding why the cost function $J = \int_0^\infty (x^\top Qx + u^\top Ru) dt$ is quadratic. It is not arbitrary quadratic is the unique choice that is always non-negative, differentiable at zero, and leads to a closed-form solution. Any other norm either loses one of these properties or destroys analytical tractability.

The derivation of CARE from the HJB equation made the whole pipeline click. The key realization was that you never solve the infinite integral directly. Instead, by guessing $V(x) = x^\top Px$ (motivated by the quadratic structure of the problem), the infinite optimization collapses into a single algebraic equation. Finding P once gives you $K = R^{-1}B^\top P$ a constant gain that encodes the entire optimal infinite-horizon strategy. The infinity disappears not because you ignore it but because a stable system’s cost integrand decays to zero.

Bryson’s rule for tuning Q and R taught me something practical: normalization by maximum acceptable values is not just a heuristic. It removes units and puts all states on equal footing, so the relative weights in Q reflect actual physical priorities rather than accidents of scale.

Kalman Filter and State Estimation

The Kalman filter reframed estimation in a way that connected directly to the LQR derivation. Both solve an optimization problem; both yield a gain from a Riccati equation; both compress infinite information into a single matrix. The difference is that LQR optimizes over a future trajectory while the Kalman filter optimizes over current uncertainty.

The most clarifying moment was understanding how $\dot{\phi}$ is recovered despite never being measured. Column 2 of C is identically zero the filter receives no direct information about wheel velocity. Yet the predict step propagates $\dot{\phi}$ forward through A , and the innovation ($z_t - C\bar{\mu}_t$) indirectly corrects it via the dynamics coupling. This made observability concrete: full rank of the observability matrix is not a formality but a guarantee that unmeasured states leave a detectable fingerprint in what you do measure.

The distinction between Q_f (process noise) and R_f (measurement noise) as competing forces on the Kalman gain was also important. Large Q_f means “I distrust my model” and pushes the filter toward trusting sensors. Large R_f means “I distrust my sensors” and pushes it toward trusting the dynamics. The gain K_t^{KF} sits at the optimal balance and unlike LQR, this balance has a direct closed-form because the minimization is linear in the gain.

iLQR and Nonlinear Control

iLQR clarified what makes nonlinear control hard and what the simplest principled response to that hardness looks like. The TWIP’s linearized model has a 50% error at $\theta = 1.5$ rad ($\sin(1.5) = 0.997$ vs the linearization’s $\theta = 1.5$). LQR cannot know this and consequently overestimates the restoring force, producing an overly aggressive response. iLQR simply re-linearizes at every point along the current trajectory rather than at the fixed equilibrium the same LQR machinery, applied iteratively. The result is a time-varying K_k and a feedforward d_k that together account for the true nonlinear geometry.

The practical lesson was that for a well-designed system near its operating point, the linear model is often more accurate than expected. LQR and iLQR are visually indistinguishable at small angles and differ only modestly at large ones. This does not diminish iLQR it validates the linearization approach and shows that the 6% cost reduction at large angles is the appropriate correction, not a radical departure.

Trajectory Following

Changing the cost from $x^\top Q x$ to $(x - x^*)^\top Q (x - x^*)$ with $x^* = [2\pi, 0, 0, 0]^\top$ was a small modification with a large conceptual payoff: it showed that the same framework handles both regulation and trajectory following. The feedforward term d_k from iLQR became $u_{ff}(t)$ naturally, and the time-varying gain K_k became $K_{\text{special}}(t)$ without any additional machinery. The cost function encodes the goal; the algorithm finds the policy.

Broader Takeaway

The through-line of this assignment is that optimal control is a consistent philosophy: encode what you care about as a scalar cost, use dynamic programming or calculus of variations to derive what the optimal solution must look like, then solve the resulting equations. LQR, the Kalman filter, and iLQR are all instances of this same pattern at different levels of complexity. Understanding

the pattern not just the formulas is what makes it possible to extend each method and recognize when it applies.

My overall take away is that now I have good mental map of controls, I came right from undergrad with absolutely no background in controls, from this point I have a decent grasp of the field as a whole and what parts fit where, I am comfortable enough with the stuff now that at this point, I should be able to pick up a book and fill my gaps in knowledge as needed, and actually be able to apply this stuff to things in real life.

7 Summary

Part	Method	Key Result
1	LQR via CARE	Stable balance from $\theta_0 = 0.1$ rad in ≈ 2 s
2	Kalman Filter via DARE	$\dot{\phi}$ recovered without direct measurement
3	iLQR	6% cost reduction at large angles; visible difference at $\theta_0 = 1.5$ rad
4	iLQR + modified cost	$\phi \rightarrow 2\pi$, $\theta \approx 0$ throughout

All code implemented in Python using SymPy (symbolic dynamics and linearization), NumPy/SciPy (CARE, DARE, integration), and Matplotlib (visualization and animation). Single Jupyter notebook with one cell per module.